```
%regQ = deepgen.alloc_buffer {scope = "local"} -> memref<2x2xf32>
%regK = deepgen.alloc_buffer {scope = "local"} -> memref<2x4xf32>
...
// Stage: Prologue
affine.for %i = 0 to 2 {
  %0 = affine.load %smemQ[%k, map1(%tid, %i)] : memref<128x32xf32>
  affine.store %0, %regQ[0, %i] : memref<2x2xf32>
}
affine.for %i = 0 to 4 {
  %0 = affine.load %smemK[0, map2(%tid, %i)] : memref<2x16x32xf32>
  affine.store %0, %regK[0, %i] : memref<2x4xf32>
}
// Stage: Main Loop
affine.for %bk = 1 to 16 {
  affine.for %i = 0 to 2 {
    %0 = affine.load %smemQ[%k + %bk, map1(%tid, %i)] : memref<128x32xf32>
    affine.store %0, %regQ[%bk mod 2, %i] : memref<2x2xf32>
  }
  affine.for %i = 0 to 4 {
    %0 = affine.load %smemK[%bk, map2(%tid, %i)] : memref<2x16x32xf32>
    affine.store %0, %regK[%bk mod 2, %i] : memref<2x4xf32>
  }
  affine.for %i = 0 to 2 {
    affine.for %j = 0 to 4 {
      %0 = affine.load %regQ[(%bk + 1) mod 2, %i] : memref<2x2xf32>
      %1 = affine.load %regK[(%bk + 1) mod 2, %j] : memref<2x4xf32>
      %2 = affine.load %acc_s[%i, %j] : memref<2x4xf32>
      %3 = arith.mulf %0, %1 : f32
      %4 = arith.addf %2, %3 : f32
      affine.store %4, %acc_s[%i, %j] : memref<2x4xf32>
    }
  }
}
// Stage: Epilogue
affine.for %i = 0 to 2 {
  affine.for %j = 0 to 4 {
    %0 = affine.load %regQ[1, %i] : memref<2x2xf32>
    %1 = affine.load %regK[1, %j] : memref<2x4xf32>
    %2 = affine.load %acc_s[%i, %j] : memref<2x4xf32>
    %3 = arith.mulf %0, %1 : f32
    %4 = arith.addf %2, %3 : f32
    affine.store %4, %acc_s[%i, %j] : memref<2x4xf32>
  }
}
```